> **Current way (2026-10-22):** the whole live-stacking system now comes up with `tools/local_night.py` in the Starfront repo (server, hub, worker), and the hub is operated with Rapid Stacker Collab's `docs/runbooks/starfront-hub.md`. What follows is the older, manual route.

Running the servers and website locally

How to run the two systems live stacking touches on this Mac, with no cloud account, and reach them from N.I.N.A. on the Windows PC. Written 2026-10-16, for Starfront plugin 1.0.0.32. The Starfront launcher in section 1 was run and its sign-in checked end to end (`tools/check_local_server.py`). The Rapid Stacker Collab commands were read from that project's run instructions, and that stack was not started while this was written.

System	What runs	Address on this Mac
Starfront collaboration server	Python (FastAPI); also serves the files page and the plugin feed	`http://127.0.0.1:8800`
Rapid Stacker Collab backend and website	One Node process: the real Lambda code over in-memory stand-ins for AWS, plus the static website	`http://127.0.0.1:8787`
Rapid Stacker Collab worker (optional)	The Rust daemon that does the stacking	none; it talks to the stack above

Two things need care. Signing in to the Starfront server normally goes through Discord. And both servers listen on this Mac only by default, so N.I.N.A. on the Windows PC needs a way in.

1. The Starfront collaboration server

The server's packages (FastAPI, uvicorn) are in both the default `python3` on this Mac (`/opt/anaconda3`) and `/usr/local/bin/python3`; Homebrew's `/opt/homebrew/bin/python3` has none of them. Start the server with its local launcher:

```
cd ~/Develop/EKEServices/Astro/starfront
/usr/local/bin/python3 server/run_local.py
```

It listens on `http://127.0.0.1:8800` and keeps its database and admin token in `~/starfront-local`, apart from your real settings. `--port` and `--data` change them. It is the ordinary server (`server/run.py`) with one difference: signing in.

Signing in

The N.I.N.A. plugin and the Starfront program sign in only through Discord, and a real Discord application is a lot to set up for a test. The launcher brings a stand-in Discord of its own. Signing in from the plugin or the program opens a page that asks only for a name, with **ncedelliott** filled in:

- **ncedelliott owns the server.** It can start collaborations and do anything the admin token can. `--name` makes someone else the owner.

- **Any other name typed there** signs in as an ordinary member, which is how to see what somebody else's telescope sees.
- **The same name is the same person every run**, so its collaborations are still there after a restart.

The stand-in signs in anyone who asks, so by default the launcher listens on this Mac only (it has no `--host`). Since 2026-10-22, `--lan` makes it listen on the whole network (0.0.0.0) and hand out this Mac's LAN address, so the Windows PC can reach it directly without the tunnel; anyone on that network can then sign in as anyone, and `--lan` is refused unless `--data` is a folder of its own. The launcher also refuses a folder holding a `discord.env` and the folder a real server uses by default, and it ignores any Discord settings exported in the shell. The page is served by the server itself, under `/_local/discord`, so the tunnel in section 3 needs nothing extra.

A real Discord application instead. To test against Discord itself, start the ordinary server with `STARFRONT_DATA=~/starfront-local /usr/local/bin/python3 server/run.py`, create an application in the Discord developer portal, register the redirect `http://127.0.0.1:8800/auth/discord/callback`, and put a `discord.env` in `~/starfront-local`:

```
STARFRONT_DISCORD_CLIENT_ID=<client id>
STARFRONT_DISCORD_CLIENT_SECRET=<client secret>
STARFRONT_DISCORD_GUILD=<your server's id>
STARFRONT_DISCORD_OWNERS=<your Discord user id>
STARFRONT_PUBLIC_URL=http://127.0.0.1:8800
```

The startup line says `discord sign-in on` when it has taken. Whether Discord accepts a plain-http `127.0.0.1` redirect has not been confirmed; if it refuses, use `localhost` in both places. `run_local.py` will not start on a folder with a `discord.env` in it, so keep the two apart.

2. The Rapid Stacker Collab backend and website

The repository has a "local stack": one Node process that runs the project's real Lambda handlers over in-memory stand-ins for DynamoDB, S3, SQS, SSM and SES. It serves the API and the website from one address and cannot reach AWS. The full manual is `docs/runbooks/local-stack.md`.

```
cd ~/Develop/EKEServices/Astro/Rust/rapid-stacker-collab/backend
bash scripts/clean.sh
node local/server.mjs --port 8787 --site ../website --state ~/collab-local
```

`scripts/clean.sh` is needed because the stack refuses to start while `build.sh` has left `node_modules` folders inside the Lambdas. `backend/node_modules` itself is already installed here (`npm ci` installs it otherwise).

Then:

1. Open `http://127.0.0.1:8787` and register an account.
2. Nothing is e-mailed. The verification link is captured at `http://127.0.0.1:8787/_local/mail?to=<your address>`.
3. Create a project. Its join code is what goes on the Starfront collaboration's form.

Option	Meaning
`--port N`	Listen port on 127.0.0.1 (default 8787). The stack never binds another address.
`--state DIR`	Keep uploaded files, stacks and previews under `DIR` across restarts.
`--site DIR`	Also serve the website at `/`.
`--public-directory`	List public projects on the home page (production keeps this off).

Things to know:

- **Accounts, projects and queues live in memory** and are gone when the stack stops. `--state` keeps only the files: uploads, stacks, previews.
- **Demo data.** `node local/drive.mjs --base http://127.0.0.1:8787 --worker` creates an owner and two contributors, uploads frames, and runs a stand-in worker. The stand-in moves frames along but does not really stack them.
- **Node version.** This Mac has Node 26; the runbooks say Node 22. It will probably work. If the stack misbehaves, try Node 22 first.

Actually stacking: the worker

Uploaded frames stay at "uploaded" until the Rust worker daemon (`collab-worker run`) works on them. Against the local stack it takes its settings from `GET /_local/worker-env`, plus a scratch folder and a star catalogue. `docs/runbooks/local-e2e.md` documents this, and the quickest way to see the whole loop work is the script that builds and runs it:

```
cd ~/Develop/EKEServices/Astro/Rust/rapid-stacker-collab
scripts/local-e2e.sh --only one
```

Building the worker needs access to the private `rapid-stacker` repository (with `CARGO_NET_GIT_FETCH_WITH_CLI=true`). Real frames need the star catalogue copy at `~/Astronomy/XPSD`; synthetic frames use `COLLAB_LOCAL_CATALOG=synthetic`.

3. Reaching both from N.I.N.A. on the Windows PC

The Collab stack listens on 127.0.0.1 only, by design and with no option to change it. The Starfront server is the same unless started with `--host 0.0.0.0` (`server/run.py`) or `--lan` (`server/run_local.py`, section 1). The clean answer for both is an SSH tunnel from the Windows PC:

1. On the Mac, turn on Remote Login: System Settings > General > Sharing.
2. On Windows, which has an SSH client built in, keep this running in a PowerShell window:

```
ssh -N -L 8800:127.0.0.1:8800 -L 8787:127.0.0.1:8787 eddie@<mac-name>.local
```

Both servers then appear at `127.0.0.1` on the Windows PC as well. That matters, because the links the servers generate point at `127.0.0.1`: the Discord callback, the Collab approval page, the verification links. Through the tunnel they work in the Windows browser exactly as on the Mac.

4. Pointing the plugin at them

What	How
Live stacking	Options > Plugins > Starfront > *Rapid Stacker Collab server*: `http://127.0.0.1:8787/api/`
Starfront server	No box since plugin 1.0.0.26. Close N.I.N.A., edit the settings file below, set `"ServerUrl": "http://127.0.0.1:8800"`. Set it back to `""` to return to collab.starfront.space.

The settings file is:

```
%LOCALAPPDATA%\NINA\PluginSettings\896B4E28-7CEB-4731-BB97-8BADE7460268\settings.json
```

Then:

- **Sign in again on the local servers.** Tokens from the real servers mean nothing to the local ones, so sign out of both first.
- **The Rapid Stacker Collab approval page opens by itself**, on `http://127.0.0.1:8787/device` with the code filled in. The plugin opens https pages, and plain http only on this PC's own address (`127.0.0.1`, `localhost`, `::1`), which is where the tunnel puts the stack. Development builds from before this change opened https pages only; with one of those, open the page by hand and type the code the options page shows.
- **Use the simulator camera** for a first run, so a night of frames costs nothing.

5. A first end-to-end run

1. Start the Starfront server with `server/run_local.py` (section 1) and the Collab stack (section 2).
2. Open the tunnel from Windows (section 3) and point the plugin at both servers (section 4).
3. On the Collab website, register, verify, and create a project. Copy its join code.
4. In N.I.N.A., sign in to Starfront (Continue as ncedelliott), start a collaboration on the Starfront panel, and paste the join code into its form.
5. Turn on *Live stacking*, sign in to Rapid Stacker Collab, and wait for a check-in (or sign in again, which checks in). *Linked collaborations* should show *Joined*.
6. Put the *Starfront collaboration* container in a sequence and run it for a few frames. The panel's *Live stacking* block should count them as waiting, then sent.
7. With the worker running, the project's page on the Collab website shows the stack growing.

`nina/MANUAL_CHECKLIST.md` in the Starfront repository is the full list of what to check, with the expected result of each step.

6. Shutting down

- Stop each server with Ctrl+C in its terminal, and close the SSH window on Windows.
- Set the plugin's `ServerUrl` back to `""` and the Collab server address back to its default (blank it), then sign in to the real servers again.
- `~/starfront-local` and `~/collab-local` hold everything the local runs wrote; delete them when finished.